

---

# Projet RoboCup Rescue

## Robot Management System (RMS)

---

### Interface IHM Web

Développement en deux étapes

*Contrôle du robot · Bras · LIDAR · Caméra*

**Étape 1 — Contrôle du Robot et du Bras**

*SERRHINI Abdellah*

**Étape 2 — Visualisation LIDAR et Caméra**

*Mounih Mohamed*

**Formation :** IATIC4

**Université :** Université de Versailles Saint-Quentin-en-Yvelines

**Encadrant :** Rodolphe Thiénard

**Année :** 2024 – 2025

---

# Table des matières

# Introduction générale

Le projet RoboCup Rescue vise à développer un robot mobile semi-autonome capable d'intervenir dans des environnements sinistrés pour localiser et assister des victimes. Ce robot, baptisé *RMS (Robot Management System)*, est construit autour de la plateforme ROS 2 Jazzy et intègre plusieurs modules spécialisés : navigation LIDAR, bras robotique, caméra, accéléromètre et interface de pilotage à distance.

Ce rapport se concentre sur la **partie Interface Homme-Machine (IHM)**, développée sous forme d'une application web légère, accessible depuis n'importe quel navigateur sur le réseau local du robot.

Le développement de cette interface s'est déroulé en **deux étapes complémentaires** :

- **Étape 1** (SERRHINI Abdellah) : mise en place du contrôle du robot et du bras robotique. Cette première version de l'interface permet la téléopération complète du robot : déplacement, flippers, orientation de la caméra et contrôle du bras via des sliders.
- **Étape 2** (Mounih Mohamed) : enrichissement de l'interface avec la visualisation du LIDAR en temps réel et l'intégration du flux vidéo de la caméra.

L'interface repose sur le protocole **rosbridge** : un serveur WebSocket exposant les topics ROS 2 du robot, auxquels le navigateur se connecte via la bibliothèque JavaScript `roslibjs`.

# Chapitre 1

## Architecture de l'interface IHM

### 1.1 Contexte technologique

L'interface IHM est une application web statique (HTML/CSS/JavaScript). Elle ne nécessite aucun framework ni langage serveur; elle fonctionne entièrement dans le navigateur et communique avec le robot via une connexion WebSocket.

#### Stack technologique

- **Frontend** : HTML5, CSS3, JavaScript vanilla
- **Bibliothèque ROS** : `roslibjs` + `EventEmitter2`
- **Serveur WebSocket** : `rosbridge_server` (ROS 2)
- **Serveur de fichiers** : `python3 -m http.server 8080`
- **Branche Git** : `interface` / dossier `IHM_Robot_RMS/`

### 1.2 Principe de communication

La chaîne de communication complète est la suivante :

Navigateur *WebSockets* : `//ip:9090` `rosbridge_server` *DDS* Nodes ROS 2 →  
Hardware

- Le navigateur publie des commandes (joystick, sliders) sur des topics ROS 2.
- Il s'abonne aussi à des topics de retour (télémétrie, images LIDAR, flux caméra).
- **L'IP du robot est configurable** depuis l'interface et persistée en `localStorage`.

## 1.3 Structure des fichiers

| Fichier             | Rôle                                    |
|---------------------|-----------------------------------------|
| index.html          | Dashboard principal (4 panneaux iframe) |
| deplacement_robot.* | Téléopération du robot (étape 1)        |
| deplacement_bras.*  | Contrôle du bras (étape 1)              |
| vision_lidar.*      | Visualiseur LIDAR Canvas (étape 2)      |
| vision_camera.*     | Flux caméra temps réel (étape 2)        |
| *.css               | Styles par page                         |

TABLE 1.1 – Organisation des fichiers de l'interface IHM

## Chapitre 2

# Étape 1 — Contrôle du Robot et du Bras

**Développeur :** SERRHINI Abdellah

**Objectif :** Permettre le pilotage complet du robot (déplacement, flippers, caméra orientable) et la commande du bras robotique depuis l'interface web.

## 2.1 Déplacement du robot

### 2.1.1 Interface graphique

La page `deplacement_robot.html` présente deux zones :

- **Zone gauche** : affichage de la télémétrie du robot (vitesses, inclinaison, boussole).
- **Zone droite** : boutons directionnels (avant, reculer, gauche, droite, stop), contrôle des flippers et de la caméra orientable.

### 2.1.2 Topic ROS 2 utilisé

Toutes les commandes de déplacement sont publiées sur un unique topic :

| Topic                 | Type                         | Direction |
|-----------------------|------------------------------|-----------|
| <code>/joy_ihm</code> | <code>sensor_msgs/Joy</code> | Publié    |

TABLE 2.1 – Topic de commande du déplacement

Le message Joy contient :

- `axes[1]` : avancer (+1.0) / reculer (-1.0)
- `axes[0]` : gauche (+1.0) / droite (-1.0)
- `axes[2]` : flipper avant-gauche ( $\pm 1.0$ )

- **buttons[1]** : flipper avant-droite
- **buttons[2]** : flipper arrière
- **buttons[5]** : caméra haut
- **buttons[7]** : caméra bas

```

1 var topic = new ROSLIB.Topic({
2   ros      : ros,
3   name     : '/joy_ihm',
4   messageType : 'sensor_msgs/Joy'
5 });
6
7 function deplacement_avant() {
8   var message = new ROSLIB.Message({
9     axes: [0.0, 1.0, 0.0, 0.0, 0.0, 0.0],
10    buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
11  });
12  topic.publish(message);
13 }

```

Listing 2.1 – Envoi du message de déplacement avant sur /joy\_ihm

### 2.1.3 Contrôle clavier

En complément des boutons graphiques, un contrôle clavier AZERTY est implémenté :

| Touche | Action                |
|--------|-----------------------|
| Z      | Avancer               |
| S      | Reculer               |
| Q      | Gauche                |
| D      | Droite                |
| A      | Flipper AVG monter    |
| E      | Flipper AVG descendre |
| M / P  | Caméra haut / bas     |
| Espace | Stop                  |

TABLE 2.2 – Affectation des touches clavier

#### Sécurité anti-dérive

À chaque événement **keyup**, la commande **stop()** est automatiquement envoyée. Cela garantit que le robot s'arrête dès que l'opérateur relâche une touche, évitant tout dérive involontaire.

### 2.1.4 Télémétrie en temps réel

Cinq subscribers ROS 2 affichent les données robot en temps réel dans le DOM :

```

1 var subscriber1 = new ROSLIB.Topic({
2   ros: ros, name: '/vitesse1', messageType: 'std_msgs/Int32'
3 });
4 subscriber1.subscribe(function(message) {
5   document.getElementById('vitesse1').innerHTML = message.data;
6 });
7
8 var subscriber2 = new ROSLIB.Topic({
9   ros: ros, name: '/vitesse2', messageType: 'std_msgs/Int32'
10 });
11 subscriber2.subscribe(function(message) {
12   document.getElementById('vitesse2').innerHTML = message.data;
13 });
14 // ... (identique pour inclinaison1, inclinaison2, boussole)

```

Listing 2.2 – Subscriber correct — chaque variable reçoit ses propres données

#### Bug critique corrigé

Dans la version initiale, les subscribers 2 à 5 appelaient tous `subscriber1.subscribe()` au lieu de leur propre méthode. Seul le premier subscriber recevait des données; les champs `vitesse2`, `inclinaison1`, `inclinaison2` et `boussole` restaient vides. **Chaque subscriber appelle désormais sa propre méthode `.subscribe()`.**

## 2.2 Contrôle du bras robotique

### 2.2.1 Interface graphique

La page `deplacement_bras.html` présente :

- **6 sliders** correspondant aux 6 degrés de liberté du bras.
- **3 boutons** de positions pré-définies : Rangement, Déploiement, Position haute.



## 2.2.2 Description des sliders

| Axe | Plage       | Centre | Mouvement                   |
|-----|-------------|--------|-----------------------------|
| X   | 1600 – 2400 | 2048   | Translation axe 1           |
| Y   | 1400 – 2400 | 2048   | Translation axe 2           |
| Z   | 0 – 1       | 0.5    | Translation axe 3 (binaire) |
| RX  | 0 – 4095    | 2048   | Rotation axe 4              |
| RY  | 700 – 3200  | 2048   | Rotation axe 5              |
| RZ  | –180 – +180 | 0      | Rotation axe 6 (degrés)     |

TABLE 2.3 – Paramètres des sliders de contrôle du bras

## 2.2.3 Topic ROS 2 utilisé

| Topic      | Type                     | Direction |
|------------|--------------------------|-----------|
| /choix_pos | std_msgs/Int16MultiArray | Publié    |

TABLE 2.4 – Topic de commande du bras

```

1 function sendSliderValues() {
2   var message = new ROSLIB.Message({
3     layout : { dim: [], data_offset: 0 },
4     data   : [
5       sliderValues.dx,      // axe X
6       sliderValues.dy,      // axe Y
7       sliderValues.dz,      // axe Z
8       sliderValues.rx,      // rotation X
9       sliderValues.ry,      // rotation Y
10      sliderValues.rz,       // rotation Z
11      sliderValues.bouton    // 0=manuel, 1=rangement, 2=
                             //   deploiement, 3=haut
12    ]
13  });
14  topic.publish(message);
15 }

```

Listing 2.3 – Publication du message de contrôle du bras

Les positions pré-définies sont gérées par la fonction :

```

1 function setButtonValueAndSend(buttonValue) {
2     sliderValues.bouton = buttonValue;
3     sendSliderValues();
4 }
5 // Appel : setButtonValueAndSend(1) => rangement
6 //         setButtonValueAndSend(2) => deploiement
7 //         setButtonValueAndSend(3) => position haute

```

Listing 2.4 – Activation d’une position pré-définie

## 2.3 Dashboard — Première version

Dans la première étape, le fichier `index.html` est un dashboard simplifié présentant les pages de contrôle du robot et du bras sous forme d’iframes. L’opérateur peut :

- Configurer l’IP du robot dans la barre supérieure (stockage en `localStorage`).
- Passer d’une page à l’autre via les boutons de navigation.

### Résultats de l’étape 1

À l’issue de la première étape, l’opérateur peut piloter le robot en déplacement et en rotation, contrôler les flippers, orienter la caméra et commander le bras robotique depuis n’importe quel navigateur connecté au réseau du robot.

## Chapitre 3

# Étape 2 — Visualisation LIDAR et Caméra

**Développeur :** Mounih Mohamed

**Objectif :** Enrichir l'interface avec la visualisation des données LIDAR en temps réel sur un canvas HTML5 et l'intégration du flux vidéo de la caméra.

### 3.1 Visualisation LIDAR

#### 3.1.1 Contexte

Le robot est équipé d'un LIDAR 2D (Hokuyo UST-10LX) publiant ses mesures sur le topic `/scan`. L'objectif est d'afficher ces données sous forme d'une vue polaire en temps réel dans le navigateur, sans aucun plugin externe.

#### 3.1.2 Architecture de la solution

La solution repose entièrement sur l'API Canvas d'HTML5 :

1. Un subscriber `roslibjs` s'abonne au topic `/scan`.
2. À chaque message reçu, les données polaires (distance, angle) sont converties en coordonnées cartésiennes et dessinées sur le canvas.
3. La vue se rafraîchit automatiquement à chaque nouvelle donnée LIDAR.

| Topic              | Type                               | Direction |
|--------------------|------------------------------------|-----------|
| <code>/scan</code> | <code>sensor_msgs/LaserScan</code> | Abonné    |

TABLE 3.1 – Topic LIDAR utilisé

### 3.1.3 Algorithme de rendu

L'algorithme de projection polaire vers cartésien est le suivant :

```
1 function drawScan(ranges, angleMin, angleIncrement, rangeMax) {  
2   drawBackground(); // reinitialise le canvas  
3  
4   var cx = W / 2, cy = H / 2;  
5   var maxR = Math.min(W, H) / 2 - 10;  
6   var scale = maxR / rangeMax;  
7  
8   for (var i = 0; i < ranges.length; i++) {  
9     var r = ranges[i];  
10    // Filtrage : ignorer NaN, Inf et hors portee  
11    if (!isFinite(r) || r <= 0 || r > rangeMax) continue;  
12  
13    var angle = angleMin + i * angleIncrement; // radians  
14    // Projection polaire -> cartésien (axe Y du canvas inverse)  
15    var x = cx + Math.sin(angle) * r * scale;  
16    var y = cy - Math.cos(angle) * r * scale;  
17  
18    // Couleur dependante de la distance  
19    // (rouge = proche, vert = loin)  
20    var ratio = r / rangeMax;  
21    var red = Math.round(255 * (1 - ratio));  
22    var green = Math.round(255 * ratio);  
23    ctx.fillStyle = 'rgb(' + red + ',' + green + ',100)';  
24    ctx.fillRect(x - 2, y - 2, 4, 4);  
25  }  
26 }
```

Listing 3.1 – Rendu LIDAR — projection polaire vers cartésien

### 3.1.4 Éléments visuels du canvas

| Élément               | Description                                       |
|-----------------------|---------------------------------------------------|
| Fond noir             | #0a0a1a pour un contraste maximal                 |
| Cercles concentriques | 4 cercles de distance : 2.5 m, 5 m, 7.5 m, 10 m   |
| Croix de quadrant     | Axes avant/arrière/gauche/droite                  |
| Point central         | Position du robot (cyan)                          |
| Points de mesure      | Couleur interpolée rouge → vert selon la distance |
| Labels directionnels  | AVANT, ARRIÈRE, G, D                              |

TABLE 3.2 – Éléments graphiques du visualiseur LIDAR

```

1 var scanSubscriber = new ROSLIB.Topic({
2   ros      : ros,
3   name     : '/scan',
4   messageType : 'sensor_msgs/LaserScan',
5   throttle_rate : 100 // max 10 Hz pour le rendu
6 });
7
8 scanSubscriber.subscribe(function(msg) {
9   drawScan(
10    msg.ranges,
11    msg.angle_min,
12    msg.angle_increment,
13    Math.min(msg.range_max, 10.0)
14  );
15  // Mise à jour du compteur de points
16  var valid = msg.ranges.filter(function(r) {
17    return isFinite(r) && r > 0 && r <= msg.range_max;
18  }).length;
19  document.getElementById('lidar-points').textContent = valid + '
20    pts';
  });

```

Listing 3.2 – Abonnement au topic /scan

### 3.1.5 Corrections apportées dans vision\_lidar

#### Bugs corrigés dans vision\_lidar.js et vision\_lidar.html

1. Le fichier HTML référençait `vision-lidar.js` (avec un tiret) au lieu de `vision_lidar.js` (avec un underscore). Le script n'était donc jamais chargé.
2. Le fichier JavaScript original s'abonnait à des topics de déplacement (`/deplacement_avant`, etc.) au lieu du topic `/scan`. Aucune donnée LIDAR n'était reçue.
3. L'affichage utilisait une iframe vide au lieu d'un canvas HTML5.

## 3.2 Intégration de la caméra

### 3.2.1 Contexte

La caméra du robot publie son flux vidéo sur le topic `/cv_camera/image_raw/compressed` sous forme d'images JPEG encodées en Base64. L'objectif est d'afficher ce flux en temps réel dans la page `vision_camera.html`.

| Topic                                        | Type                                     | Direction |
|----------------------------------------------|------------------------------------------|-----------|
| <code>/cv_camera/image_raw/compressed</code> | <code>sensor_msgs/CompressedImage</code> | Abonné    |

TABLE 3.3 – Topic caméra utilisé

### 3.2.2 Affichage du flux

```

1 var listener = new ROSLIB.Topic({
2   ros      : ros,
3   name     : '/cv_camera/image_raw/compressed',
4   messageType : 'sensor_msgs/CompressedImage'
5 });
6
7 listener.subscribe(function(message) {
8   var img = document.getElementById('image_sub');
9   if (img) {
10    // Decodage Base64 et affichage direct dans un element img
11    img.src = 'data:image/jpeg;base64,' + message.data;
12    // Masquer le message "En attente du flux..."
13    var nosignal = document.getElementById('cam-no-signal');
```

```
14     if (nosignal) nosignal.style.display = 'none';
15   }
16 });
```

Listing 3.3 – Réception et affichage du flux caméra

### 3.2.3 Corrections apportées dans vision\_camera

#### Bug critique corrigé — IDs dupliqués

Le fichier `vision_camera.html` original contenait **8 éléments** `<img id="image_sub">` identiques. Le DOM HTML impose l'unicité des IDs : la méthode `document.getElementById('image_sub')` ne pouvait cibler qu'un seul élément ; les 7 autres restaient invisibles. La correction a consisté à réduire le fichier à **un seul élément** `<img id="image_sub">`.

## 3.3 Dashboard final — Quatre panneaux

La deuxième étape a également abouti à la refonte complète du fichier `index.html` en un **dashboard 4 panneaux** affichant simultanément :

1. **Déplacement Robot** : contrôle directionnel et télémétrie.
2. **Vision LIDAR** : canvas de visualisation en temps réel.
3. **Caméra** : flux vidéo principal.
4. **Contrôle Bras** : sliders et positions pré-définies.

```
1 <div id="dashboard">
2   <div class="panel">
3     <div class="panel-title">Déplacement Robot</div>
4     <iframe id="frame-deplacement" src="deplacement_robot.html"
5       ></iframe>
6   </div>
7   <div class="panel">
8     <div class="panel-title">Vision LIDAR</div>
9     <iframe id="frame-lidar" src="vision_lidar.html"></iframe>
10  </div>
11  <div class="panel">
12    <div class="panel-title">Camera</div>
13    <iframe id="frame-camera" src="vision_camera.html"></iframe>
14  </div>
```

```
14 <div class="panel">
15   <div class="panel-title">Controle Bras</div>
16   <iframe id="frame-bras" src="deplacement_bras.html"></iframe>
17 </div>
18 </div>
```

Listing 3.4 – Structure du dashboard 4 panneaux (index.html)

### Résultats de l'étape 2

À l'issue de la deuxième étape, l'opérateur dispose d'un dashboard unifié offrant simultanément le pilotage du robot, la cartographie LIDAR en temps réel et la surveillance vidéo de l'environnement.



## Chapitre 4

# Synthèse des corrections et améliorations

### 4.1 Tableau récapitulatif des bugs corrigés

| Fichier                                  | Bug identifié                                        | Correction                                            |
|------------------------------------------|------------------------------------------------------|-------------------------------------------------------|
| deplacement_robot.js<br>vision_camera.js | Subscribers 2 à 5 appelaient subscriber1.subscribe() | Chaque subscriber appelle sa propre méthode           |
| vision_lidar.html                        | Référence JS incorrecte : vision-lidar.js            | Corrigé en vision_lidar.js                            |
| vision_camera.html                       | 8 éléments <img> avec le même id="image_sub"         | Réduit à 1 seul élément                               |
| deplacement_robot.html                   | Éléments DOM manquants pour la télémétrie            | Ajout des <span> vitesse1/2, inclinaison1/2, boussole |
| vision_lidar.js                          | Abonné à des topics de déplacement, pas à /scan      | Réécriture complète avec subscriber /scan             |
| vision_lidar.js                          | Affichage via iframe vide                            | Remplacé par un canvas HTML5                          |

TABLE 4.1 – Tableau récapitulatif des bugs corrigés

## 4.2 Améliorations apportées

- **Indicateurs de connexion ROS** : un badge **Connecté** / **Déconnecté** est affiché en temps réel sur toutes les pages grâce aux événements `ros.on('connection')`, `ros.on('error')`, `ros.on('close')`.
- **IP configurable dynamiquement** : l'IP du robot est saisie une seule fois dans le dashboard, stockée en `localStorage` et propagée à toutes les iframes.
- **Message d'attente** : un message informatif s'affiche tant que le flux caméra ou LIDAR n'est pas reçu.
- **Compteur de points LIDAR** : le nombre de points valides détectés par le LIDAR est affiché en temps réel.

## Chapitre 5

# Guide de déploiement

### 5.1 Lancer le serveur de fichiers

```
1 # Se placer dans le dossier de l'interface (branche : interface)
2 cd /path/to/RoboCup_ws/IHM_Robot_RMS
3
4 # Lancer le serveur local
5 python3 -m http.server 8080
6
7 # Ouvrir dans le navigateur
8 # http://localhost:8080
```

Listing 5.1 – Démarrage du serveur HTTP pour l'IHM

### 5.2 Lancer rosbridge sur le robot

```
1 # Sur le robot (ROS 2 Jazzy)
2 ros2 launch rosbridge_server rosbridge_websocket_launch.xml
3 # ou
4 ros2 run rosbridge_server rosbridge_websocket
```

Listing 5.2 – Démarrage de rosbridge\_server

### 5.3 Configuration réseau

1. Vérifier que le PC opérateur et le robot sont sur le **même réseau local**.
2. Dans la barre IP du dashboard, saisir l'**IP du robot** (ex. : 192.168.137.82).
3. Cliquer **Appliquer** : toutes les iframes reconnectent automatiquement.

### Test sans robot

L'interface peut être ouverte sans robot connecté. Les indicateurs affichent **Déconnecté** et les zones de données restent vides, mais aucune erreur JavaScript n'est générée.

# Conclusion

Ce rapport a présenté le développement de l'interface IHM web du robot RMS en deux étapes complémentaires.

La **première étape**, réalisée par SERRHINI Abdellah, a posé les bases fonctionnelles de l'interface : connexion via rosbridge, téléopération complète du robot (déplacement, flippers, caméra), contrôle du bras robotique à 6 degrés de liberté et affichage de la télémétrie en temps réel.

La **deuxième étape**, réalisée par Mounih Mohamed, a enrichi cette base avec deux modules de perception : un visualiseur LIDAR en temps réel basé sur Canvas HTML5 et l'intégration du flux caméra compressé. Ces ajouts ont transformé l'interface en un véritable poste de supervision permettant à l'opérateur d'avoir une conscience situationnelle complète pendant la mission.

L'interface, entièrement statique, légère et sans dépendance côté client, constitue un outil de contrôle robuste et extensible pour les missions RoboCup Rescue.